

Building Blocks of a GenAI Transformer

From Pre-Training to Multi-Modal Intelligence

Assignment

Student Name:	
Roll Number:	
Total Points:	

Instructions: This assignment has two halves. Part A tests your understanding of key concepts (Pre-Training, SFT, RLHF, RLVR, Vision, Function Calling). Part B is where you build something useful for yourself using Ollama on your own machine. Hints are provided throughout — use them! You may work individually or in pairs for Part B.

Part A: Conceptual Understanding (40 points)

Multiple choice and short answer questions testing your understanding of key concepts.

Question 1: Pre-Training Objectives [5 points]

Which of the following best describes Causal Language Modeling (CLM), the approach used to pre-train GPT-style models?

- A) The model is trained to predict missing words anywhere in a sentence (masked tokens).
- B) The model is trained to predict the next token based only on previous tokens (left-to-right).
- C) The model is trained to translate sentences from one language to another.
- D) The model is trained to classify entire documents into categories.

Hint: Think about the difference between GPT and BERT. GPT reads text in one direction, like reading a book from left to right. BERT can look at words from both sides. Which approach matches "causal" (cause → effect, past → future)?

Question 2: Supervised Fine-Tuning (SFT) [5 points]

After pre-training, a model can generate text but doesn't follow instructions well. Explain in 2–3 sentences what Supervised Fine-Tuning (SFT) does and why it's needed.

Hint: Consider: a pre-trained model has read billions of web pages. It knows language, but does it know how to be a helpful assistant? SFT uses instruction–response pairs to teach it.

Question 3: RLHF — Reward Modeling [5 points]

In Reinforcement Learning from Human Feedback (RLHF), human annotators rank multiple model outputs. What is the primary purpose of this ranking step?

- A) To create more training data for pre-training.
- B) To train a reward model that scores outputs based on human preferences.
- C) To directly update the model's weights through backpropagation.
- D) To remove harmful content from the training dataset.

Hint: RLHF has 3 stages: (1) SFT, (2) reward model training, (3) PPO optimization. The human rankings are used in stage 2 — they teach a separate model what "good" looks like.

Question 4: RLVR vs. RLHF [5 points]

Reinforcement Learning with Verifiable Rewards (RLVR) was used to train models like DeepSeek-R1. In 2–3 sentences, explain the key difference between RLVR and RLHF. Why might RLVR be especially useful for math and coding tasks?

Hint: In RLHF, a human (or a model trained on human preferences) decides if an answer is good. In RLVR, the reward is based on whether the answer is verifiably correct. Think: "Is $2+2=4$?" doesn't need a human to verify!

Question 5: Vision LLM Pipeline [5 points]

In a Vision LLM (like LLaVA), an image must be converted into tokens the language model can understand. What is the correct order of processing?

- A) Image → Text Description → Tokenizer → LLM
- B) Image → Patches → Vision Encoder (ViT) → Projection Layer → LLM
- C) Image → Pixel Matrix → Convolution → LLM
- D) Image → CLIP Text Encoder → Embedding → LLM

Hint: The key insight is that images and text live in different "worlds." The vision encoder converts image patches into vectors, and the projection layer translates those vectors into the LLM's language.

Question 6: Function Calling [5 points]

Modern LLMs can "call functions" (also called tool use). Explain in your own words: when an LLM "calls a function," does it actually execute code? What really happens?

Hint: Think of the LLM as a very smart dispatcher. It decides WHICH function to call and WHAT arguments to pass, then outputs a structured JSON message. Something else (your application code) actually runs the function.

Question 7: The Training Pipeline [5 points]

Below are the stages involved in creating a modern instruction-following LLM. Place them in the correct order by numbering them 1 through 5.

#	Stage	What it accomplishes (1 sentence)
---	-------	-----------------------------------

	RLHF / RLVR	
	Pre-Training (Causal LM)	
	Supervised Fine-Tuning (SFT)	
	Data Collection & Tokenization	
	Evaluation & Deployment	

Hint: Think about what each stage needs as INPUT. Pre-training needs tokenized data. SFT needs a pre-trained model. RLHF needs an SFT'd model. This gives you the order!

Question 8: Analogy Matching [5 points]

Match each concept on the left with the best real-world analogy on the right. Write the letter in the blank.

—	Concept		Analogy
—	1. Pre-Training		A. Translating a foreign book into English so you can read it
—	2. SFT		B. A student reading every textbook in the library
—	3. RLHF		C. A photo being cut into a jigsaw puzzle for analysis
—	4. Vision Encoder (ViT)		D. A teacher grading essays and saying which one is better
—	5. Projection Layer		E. An apprentice learning a specific job from a master

Hint: Start with the easiest matches. Pre-training involves learning from massive amounts of text — like reading everything in the library. The projection layer converts vision embeddings into the LLM's token space — like translating between languages.

Part B: Build Something Useful (60 points)

In this part, you'll install Ollama, explore a local LLM, and then build a tool that's genuinely useful to YOU as a student. The goal is to walk away with something you'll actually keep using.

Question 9: Set Up & Explore Ollama [10 points]

In this exercise, you will install Ollama, run a local LLM, and explore how model behavior changes with different settings. *(If you cannot install Ollama, answer the reflection questions based on what you learned in class.)*

Step 1: Install & Verify (2 pts)

Visit ollama.com and install it for your operating system. Verify the installation:

```
ollama --version
```

Paste the output here:

Step 2: Pull and Run a Model (2 pts)

Run one of the following models (pick based on your hardware):

Model	Command	Size	Best For
Llama 3.2 (1B)	ollama run llama3.2:1b	~1.3 GB	Low-end laptop
Llama 3.2 (3B)	ollama run llama3.2	~2 GB	Standard laptop
Phi-3 Mini	ollama run phi3	~2.3 GB	Standard laptop
Gemma 2 (2B)	ollama run gemma2:2b	~1.6 GB	Standard laptop

Step 3: Test 3 Prompt Types (3 pts)

Try each prompt and paste the model's response:

Prompt 1 (Factual): "What is the capital of France?"

Response:

Prompt 2 (Reasoning): "If I have 3 apples and give away 1, then buy 4 more, how many do I have? Show your step-by-step reasoning."

Response:

Prompt 3 (Creative): "Write a haiku about machine learning."

Response:

Step 4: Temperature Experiment (3 pts)

Temperature controls how "creative" vs. "deterministic" the model's output is. Use the Ollama API to compare outputs at two different temperatures. Run this Python snippet twice, changing the temperature value:

```
import requests, json

prompt = "Explain what a neural network is in exactly one sentence."

for temp in [0.0, 1.5]:
    r = requests.post("http://localhost:11434/api/generate", json={
        "model": "llama3.2", # or your model
        "prompt": prompt,
        "stream": False,
        "options": { "temperature": temp }
    })
    print(f"Temperature {temp}: {r.json()['response']}")
```

Hint: If you can't run the Python code, you can also set the temperature in the Ollama CLI. Type `/set parameter temperature 0.0` in the chat, then ask the same question. Then change to 1.5 and ask again.

Output at temperature 0.0:

Output at temperature 1.5:

What differences do you notice? Why does temperature=0 give more predictable output?

Hint: Temperature scales the logits before softmax. At temperature=0, the model always picks the highest-probability token (deterministic). At higher temperatures, lower-probability tokens get a bigger chance, making the output more varied and creative — but also potentially more random or nonsensical.

Step 5: Reflection (3 pts)

(a) Which prompt type did the model handle best? Which was hardest? Explain why, using what you know about how these models are trained.

(b) Thinking about pre-training, SFT, and RLHF — which training stage helps the model follow your instructions as a helpful assistant? Which gives it factual knowledge? Which would help it give safer, more aligned answers?

Question 10: Build: Your Personal Study Assistant [25 points]

Now build something you'll actually use. Choose **ONE** of the three projects below. Each one uses Ollama's Chat API to create a **multi-turn** tool that helps you as a student.

Option A: Flashcard Generator

Paste your class notes → the LLM generates Q&A flashcards → you can then say "harder" or "focus on chapter 3" to refine them.

Option B: Explain-Like-I'm-5 Tool

Paste a confusing paragraph → the LLM simplifies it → you can ask follow-up questions like "what does X mean?" in the same session.

Option C: Practice Quiz Maker

Give a topic → the LLM generates an MCQ quiz → you answer, and it scores you and explains what you got wrong.

Which option did you choose? (Circle one): A / B / C

(a) Design your system prompt (5 pts)

The system prompt tells the LLM how to behave. Write a system prompt for your chosen project. It should specify: the role, the expected input format, the output format (structure your output — e.g., use numbered lists, use JSON, use Q:/A: pairs), and what to do when the input is ambiguous or too short.

Hint: A good system prompt is specific. Instead of "You are helpful," try: "You are a flashcard generator. When the user pastes their notes, create exactly 5 flashcards in the format: Q: [question] / A: [answer]. Keep answers under 2 sentences. If the notes are too short, ask the user for more context before generating."

Your system prompt:

(b) Write a Python script using Ollama's Chat API (10 pts)

Your script must use the **/api/chat** endpoint (not **/api/generate**) so that the conversation has memory. Study the starter code below — it maintains a message history so the LLM remembers previous exchanges. Your job: fill in the TODOs and extend the script with your own logic.

```
import requests, json

OLLAMA_URL = "http://localhost:11434/api/chat"
MODEL = "llama3.2" # or whichever model you installed

system_prompt = """
# TODO: Paste your system prompt from part (a) here
"""

# Message history gives the model conversation memory
messages = [{"role": "system", "content": system_prompt}]

def chat(user_message):
    messages.append({"role": "user", "content": user_message})
    response = requests.post(OLLAMA_URL, json={
        "model": MODEL,
        "messages": messages,
        "stream": False
    })
    reply = response.json()["message"]["content"]
    messages.append({"role": "assistant", "content": reply})
    return reply

# TODO: Add a loop so the user can have a multi-turn conversation
# TODO: Add a way for the user to type 'quit' or 'exit' to stop
# TODO: (Option C only) Parse the quiz output and let the user
#         submit answers, then show a score
```

Requirements (your script must do all of these to earn full credit):

1. Use the **/api/chat** endpoint with message history (not **/api/generate**)
2. Support multi-turn conversation (a while loop that keeps accepting input)
3. Have a clean exit command (e.g., typing "quit" or "exit")
4. Print the model's response in a readable format
5. Handle at least one error case (e.g., what if Ollama isn't running?)

Write your complete, working Python script below (or attach a .py file)

Hint: You'll need: `pip install requests`. For error handling, wrap the API call in a try/except block. A simple way: `try: response = requests.post(...) except requests.ConnectionError: print("Error: Is Ollama running? Start it with: ollama serve")`. For multi-turn, use: `while True: user_input = input("> "); if user_input.lower() in ["quit", "exit"]: break`

(c) Test with real content — at least 3 conversation turns (5 pts)

Run your tool using actual content from one of your current courses. You must show at least 3 turns of conversation: the initial input, one follow-up that refines or builds on the output, and one more follow-up. This demonstrates that your tool supports multi-turn interaction.

Turn 1 — Your initial input:

Turn 1 — Model's response:

Turn 2 — Your follow-up (e.g., "make them harder", "explain #3 more", "focus on X"):

Turn 2 — Model's response:

Turn 3 — Your follow-up:

Turn 3 — Model's response:

Tip: The power of conversation memory means you DON'T need to re-paste your notes every time. Just say "make those flashcards harder" or "now quiz me on the same material" and the LLM already has context. Try it!

(d) Evaluate & iterate on your system prompt (5 pts)

(i) Rate the output quality across all 3 turns. Was it useful? Did the model remember context from earlier turns? What was the biggest problem?

(ii) Edit your system prompt to fix the biggest problem you identified. Show the improved prompt and re-run Turn 1 to compare the output.

Improved system prompt:

New Turn 1 output with improved prompt:

(iii) In 2–3 sentences, what did you learn about prompt engineering from this iteration?

Tip: Prompt engineering is an iterative process. If the output is too long, tell the LLM to be concise. If it's too vague, add a concrete example of what you want in the system prompt. If it hallucinates, add: "Only use information from the user's input. If you're unsure, say so."

Question 11: Build a Web UI with Token Tracking [15 points]

Your command-line study tool works, but a web interface makes it **shareable and visual**. In this exercise, you'll build a simple web frontend using **Gradio** (a Python library that creates web UIs with just a few lines of code), and add **token tracking** so you can see how much of the model's context window your conversation is using.

(a) Install Gradio and create a basic UI (5 pts)

Install Gradio, then study the starter code below. It wraps your Ollama chat function in a web interface:

```
# pip install gradio requests
import gradio as gr
import requests, json
```

```
MODEL = "llama3.2" # or your model
OLLAMA_URL = "http://localhost:11434/api/chat"

system_prompt = """Your system prompt from Q10 here"""

def chat_with_ollama(user_message, history):
    # Convert Gradio's history format to Ollama's message format
    messages = [{"role": "system", "content": system_prompt}]
    for human_msg, ai_msg in history:
        messages.append({"role": "user", "content": human_msg})
        messages.append({"role": "assistant", "content": ai_msg})
    messages.append({"role": "user", "content": user_message})

    response = requests.post(OLLAMA_URL, json={
        "model": MODEL,
        "messages": messages,
        "stream": False
    })
    reply = response.json()["message"]["content"]
    return reply

# This single line creates a full chat web interface!
demo = gr.ChatInterface(fn=chat_with_ollama, title="My Study Assistant")
demo.launch() # Opens in your browser at http://127.0.0.1:7860
```

Run this script, open the URL in your browser, and take a screenshot of your UI in action. Paste or describe what you see:

Hint: Gradio creates the entire web interface for you — text input box, send button, chat history display, and even a "Clear" button. You just provide the function that processes messages. If you get a `ConnectionError`, make sure Ollama is running (`ollama serve`).

(b) Add token counting (5 pts)

LLMs have a limited **context window** (e.g., Llama 3.2 has a 128K token window, but in practice shorter contexts give better results). Add token counting to your UI so you can monitor how much context you're using. A rough estimate: **1 token ≈ 4 characters** in English. Modify your chat function to track and display tokens:

```
def estimate_tokens(text):
    """Rough token estimate: ~4 chars per token for English text"""
    return len(text) // 4

def count_conversation_tokens(messages):
    """Count total tokens across all messages in the conversation"""
    total = 0
    for msg in messages:
```

```
        total += estimate_tokens(msg["content"])
    return total

# TODO: Call count_conversation_tokens() after each exchange
# TODO: Display the count in the response, e.g.:
#   reply += f"\n\n---\nTokens used: {tokens} / 4096 context"
# TODO: Warn the user if tokens exceed 80% of the context window
```

Show your modified code and a conversation where the token count increases over multiple turns:

Hint: The real Ollama API returns token counts in the response JSON under `eval_count` (output tokens) and `prompt_eval_count` (input tokens). For extra credit, use those instead of the estimate: `total_tokens = result.get("prompt_eval_count", 0) + result.get("eval_count", 0)`

(c) Token experiments (5 pts)

Run the following experiments and record your observations:

Experiment 1: Context window impact on speed

Have a short conversation (2 turns) and a long conversation (8+ turns) with your tool. Compare the response time for each. Does the model get slower as the conversation gets longer?

Short conversation response time:

Long conversation response time:

Your explanation (why does this happen?):

Experiment 2: Context window vs. memory

In your long conversation, reference something specific you said in Turn 1. Does the model remember it in Turn 8? Try clearing the conversation history (`messages = [system_msg]`) and asking the same question. What happens?

Did the model remember Turn 1 at Turn 8?

What happened after clearing history?

Experiment 3: System prompt token cost

Your system prompt is sent with EVERY message. Count its tokens. If your system prompt is 200 tokens and the context window is 4096 tokens, what percentage of every request is "wasted" on the system prompt? How could you make it shorter without losing its effectiveness?

System prompt token count:

Percentage of context window:

Your shortened system prompt (try to cut it by 50%):

Hint: Longer conversations = more tokens sent with each request = slower responses, because the model must process all previous messages every time. This is why real chat apps sometimes summarize older messages or use a sliding window. System prompt optimization matters: a 200-token prompt in a 4096-token window uses ~5% per turn — but it adds up fast in long conversations!

Question 12: Image Patching for Vision Models [5 points]

A Vision Transformer (ViT) divides an image into non-overlapping square patches before processing. This question connects to how multi-modal LLMs "see."

(a) An image is 224×224 pixels and uses a patch size of 16×16 . How many patches are created? Show your work.

(b) Each patch is flattened into a vector. If the image has 3 color channels (RGB), what is the dimension of each flattened patch vector?

(c) These patch vectors are then projected to a 768-dimensional embedding. A text token in BERT also has a 768-dimensional embedding. Why is it useful for image embeddings and text embeddings to have the same dimension?

Hint: For (a): divide image width by patch width, then square it: $(224/16)^2$. For (b): a 16×16 patch with 3 channels = $16 \times 16 \times 3$. For (c): think about what the projection layer does — it maps vision vectors into the same space as text tokens, so the LLM can process them together.

Question 13: Level Up: Save Your Output to a File [\[5 points\]](#)

Your study tool from Q10 prints results to the screen, but that's not very useful if you want to review later. Extend your script so it automatically saves the generated content to a file.

Starter code for file saving:

```
from datetime import datetime

def save_to_file(content, filename="study_output.txt"):
    with open(filename, "a") as f: # 'a' = append mode
        f.write(f"\n--- {datetime.now().strftime('%Y-%m-%d %H:%M')} ---\n")
        f.write(content + "\n")

# TODO: Call save_to_file() after each model response in your chat loop
# TODO: Print a confirmation like 'Saved to study_output.txt'
```

(a) Show the modified parts of your code that integrate file saving:

(b) Run your tool and paste the contents of the saved output file:

Hint: Python's `open(file, 'a')` appends without erasing previous content. For fancier output, try writing Markdown: save as `.md` and headers/bullets will render nicely in any Markdown viewer.